

SMART CAMPUS RESOURCE MANAGEMENT SYSTEM

PROJECT REPORT

CSE2006 – Programming in Java

Student: **ESHITA PARIHAR**

Project: Smart Campus Resource Management System
Java-based Command-Line Application

1. Introduction

The Smart Campus Resource Management System is a Java-based command-line application designed to manage shared college campus resources and their booking requests.

The current prototype allows users to register and log in, view resources, request bookings, detect booking conflicts, view and cancel their bookings, and report maintenance issues. The application uses Java classes, services, repositories and in-memory collections.

2. Problem Statement

Colleges manage shared resources such as classrooms, computer laboratories and seminar halls. Manual booking can cause overlapping requests, difficulty checking availability and poor record keeping.

The project provides a simple centralized command-line solution for resource viewing, booking and maintenance reporting.

3. Functional Requirements

- User registration and basic login authentication.
- Display campus resources with ID, name, type and location.
- Create booking requests with date, start time and end time.
- Detect overlapping pending bookings for the same resource and date.
- View the current user's bookings.
- Cancel the current user's pending booking.
- Report a maintenance problem with priority.
- View maintenance requests and their status.
- Exit the application safely.

4. Non-functional Requirements

- **Usability** – clear command-line menus and prompts.
- **Performance** – immediate response for the small in-memory dataset.
- **Maintainability** – separation into model, service and repository packages.
- **Reliability** – booking conflict checks occur before adding a booking.
- **Portability** – standard Java functionality allows execution on compatible systems.

5. System Architecture

The application follows a lightweight layered structure. Main handles command-line interaction, service classes contain operations and ResourceRepository manages the resource collection.

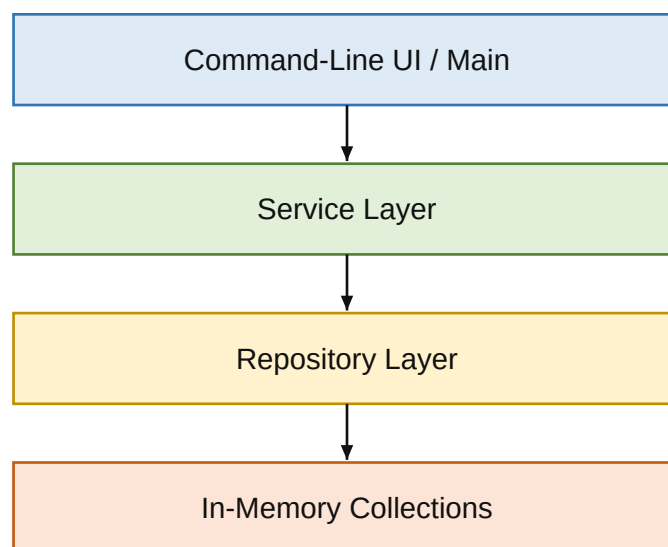


Figure 1. Current system architecture

6. Design Diagrams

6.1 Use Case Diagram

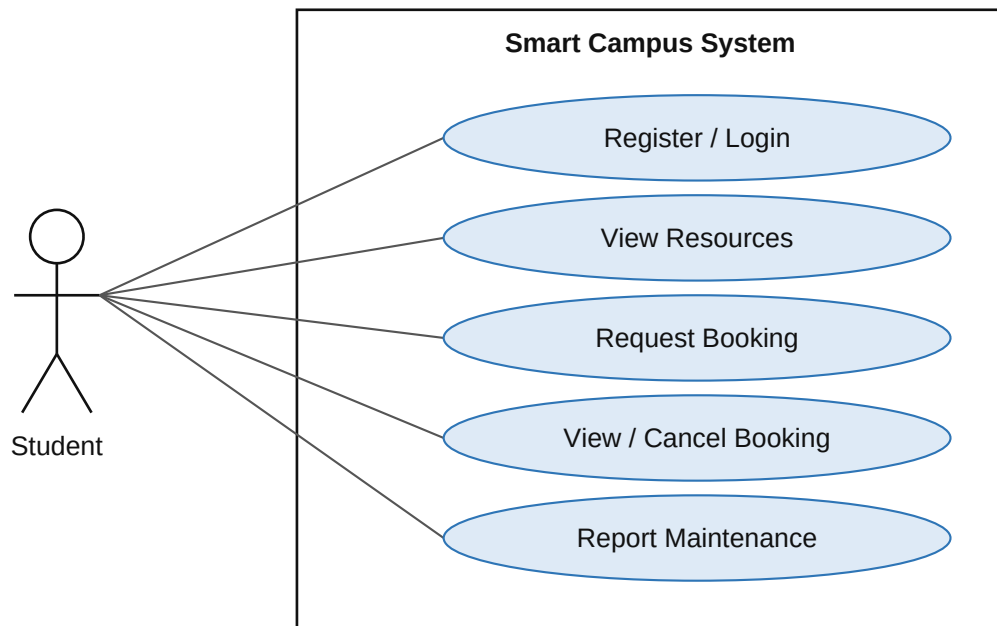


Figure 2. Implemented user-facing use cases

6.2 Workflow Diagram

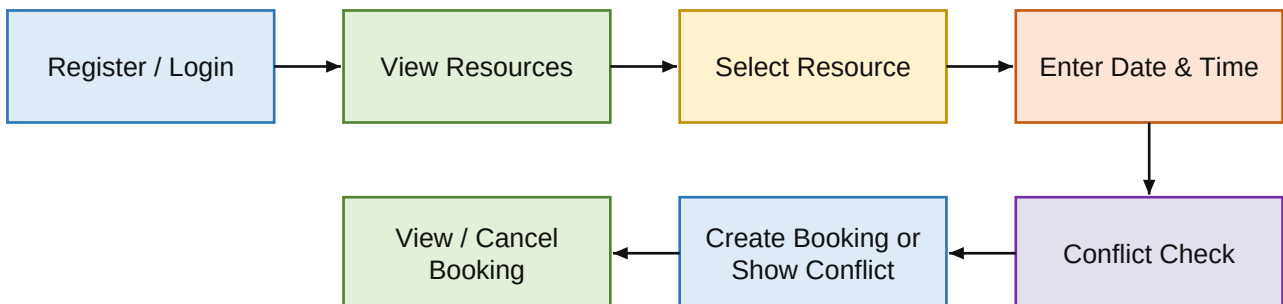


Figure 3. Booking workflow

6.3 Sequence Diagram

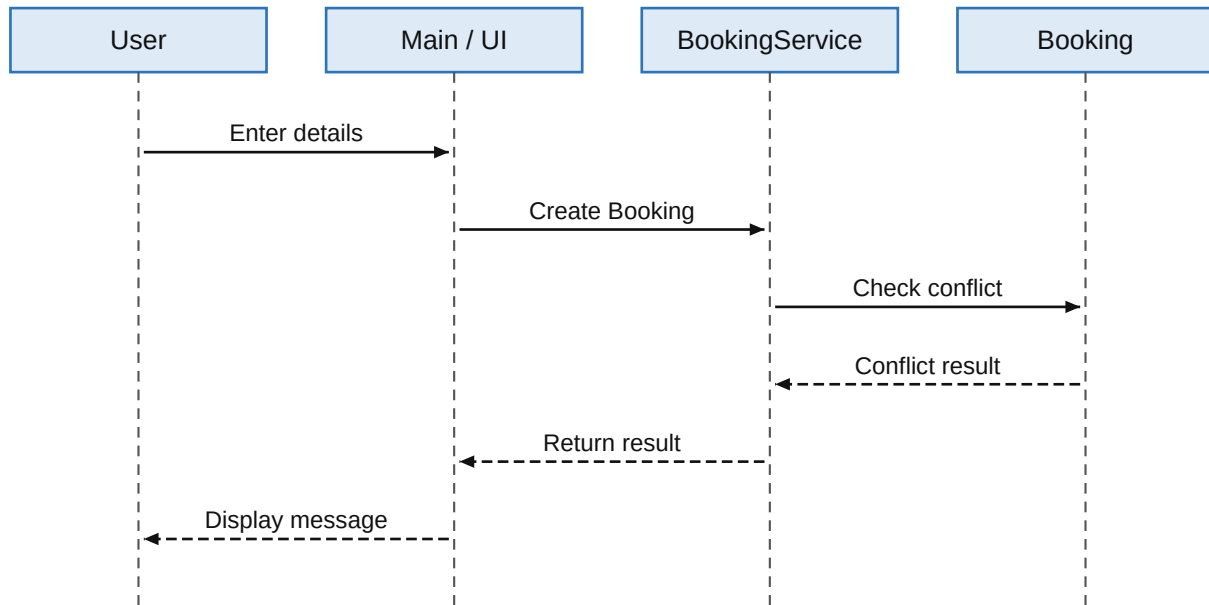


Figure 4. Booking request sequence

6.4 Class / Component Diagram

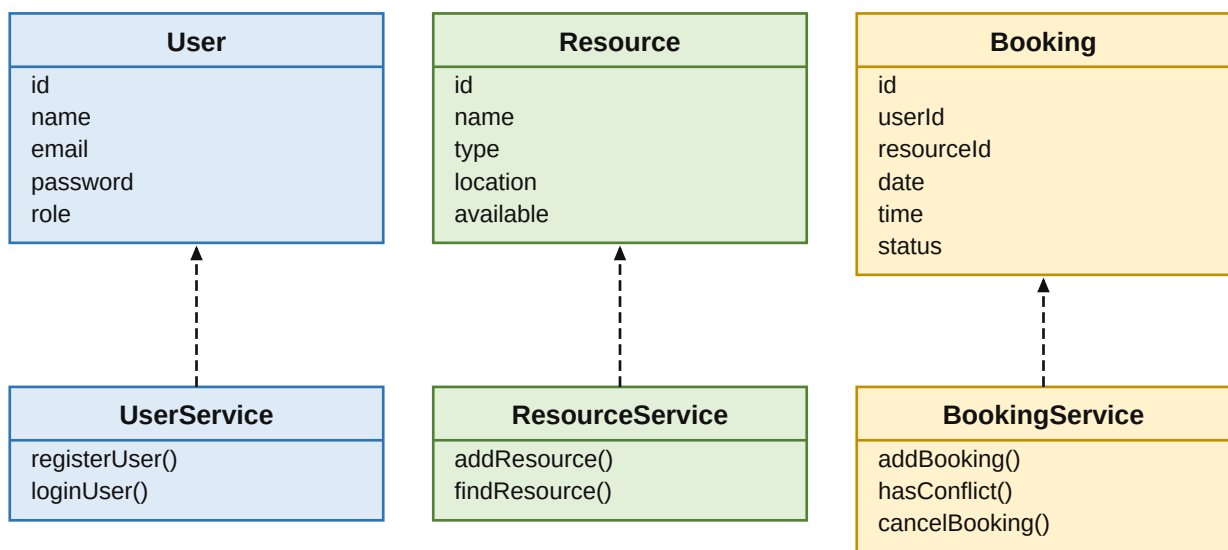


Figure 5. Simplified class/component relationships

6.5 ER Diagram

An ER diagram is not applicable to the current implementation because the running application does not persist data in a database. A DatabaseConnection class exists for possible future JDBC integration, but it is not used by the current application.

7. Design Decisions & Rationale

- A command-line interface keeps the project focused on Java programming rather than GUI development.
- ArrayList is used for simple in-memory storage appropriate to the prototype.
- Service and repository separation keeps operations modular and provides a path toward later persistence.
- Booking conflict detection uses interval overlap logic; adjacent bookings are allowed.
- Database persistence and advanced administrative functionality are documented as future enhancements rather than claimed as implemented.

8. Implementation Details

The model package contains User, Resource, Booking and MaintenanceRequest classes. UserService handles registration and login. ResourceService works with ResourceRepository. BookingService handles booking creation, conflict detection, viewing and cancellation. MaintenanceService stores and displays maintenance requests.

The booking conflict condition checks the same resource and date and verifies that the existing booking is pending. It then compares start and end times to detect overlap.

The application was compiled with javac and executed using the Java classpath command from Windows PowerShell.

9. Screenshots / Results

The working application produced the following results during manual execution.

A booking for Room 101 from 10:00 to 12:00 was successfully created. A second overlapping request was rejected by the conflict detection logic. Booking viewing and cancellation were also tested successfully.

A maintenance issue could be submitted with a resource ID, description and priority such as HIGH, and the request could then be displayed through the maintenance menu.

Resource display example:

```
----- CAMPUS RESOURCES -----  
1. Room 101 | Classroom | Block A  
2. Computer Lab 1 | Computer Laboratory | Block B  
3. Seminar Hall | Seminar Hall | Block C
```

10. Testing Approach

Test	Operation	Result
TC01	Register user	Passed
TC02	Login	Passed
TC03	View resources	Passed
TC04	Create booking	Passed
TC05	Overlapping booking	Passed
TC06	View bookings	Passed
TC07	Cancel booking	Passed
TC08	Report maintenance	Passed
TC09	View maintenance	Passed
TC10	Exit	Passed

Testing focused on the implemented command-line workflows and booking conflict behavior. The application was compiled successfully and the main functions were manually exercised.

11. Challenges Faced

- Keeping Java package declarations consistent with the source directory structure.
- Handling Scanner input when switching between numeric input and `nextLine()`.
- Implementing booking cancellation by changing the booking status.
- Designing correct time-interval overlap detection.
- Keeping the final documentation consistent with the features actually implemented.

12. Learnings & Key Takeaways

- Practiced classes, objects, constructors, encapsulation and methods.
- Applied `ArrayList` and collection iteration.
- Implemented a real-world booking conflict algorithm.
- Learned practical command-line input handling in Java.
- Used model, service and repository separation for modular design.
- Learned the importance of testing the actual application workflow.
- Gained experience preparing technical project documentation.

13. Future Enhancements

- SQLite database integration using JDBC.
- JPA/Hibernate persistence.
- Admin and Faculty roles with role-based permissions.
- Admin booking approval and rejection.

- Automated reports and text-file export.
- JUnit 5 automated tests.
- Multithreaded booking processing and synchronization.
- Web/mobile interface, email notifications and calendar integration.

14. References

- Oracle Java Documentation – Java language and API documentation.
- CSE2006 – Programming in Java course material and syllabus.
- Java documentation covering collections, input handling and object-oriented programming.
- Project source code and README.md for the Smart Campus Resource Management System.

15. Conclusion

The Smart Campus Resource Management System is a working command-line prototype for campus resource booking and maintenance reporting. It demonstrates practical Java programming through a clear user workflow, in-memory data management and booking conflict detection. The documented future enhancements provide a path toward database persistence, administration, reporting and advanced concurrency.